LoRA

Học một phần sửa nhỏ cho ma trận lớn

Phase 4 · Bài 4.4

Bộ nhớ fine-tuning gồm nhiều phần

Full fine-tuning còn giữ gradient, trạng thái optimizer và activation.

CÂU HỎI

Có thể giữ trọng số nguồn và chỉ học phần sửa không?

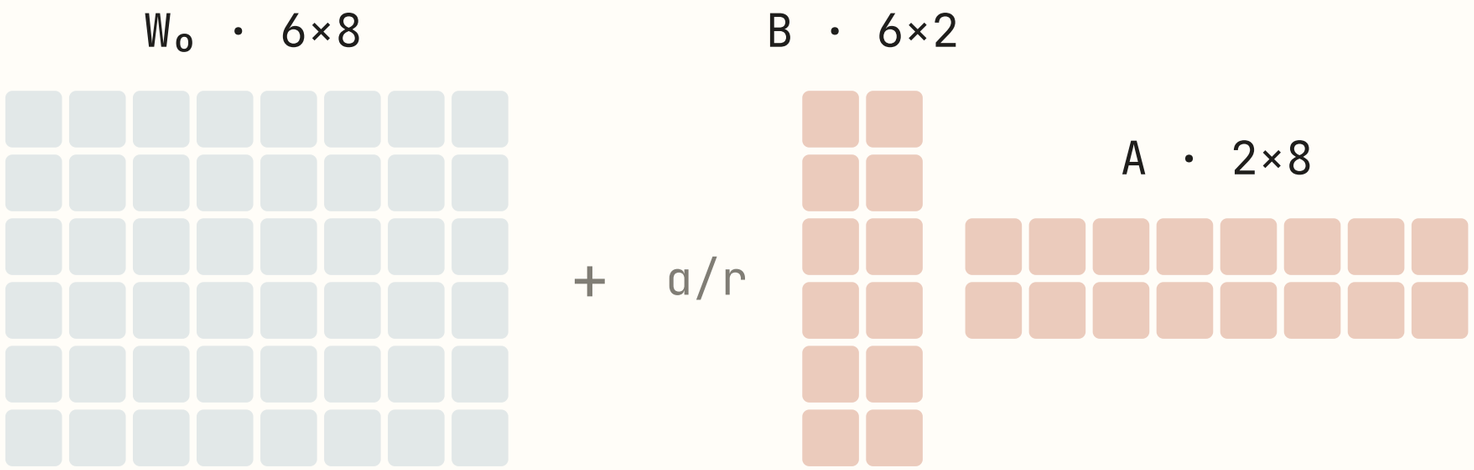
Mục tiêu buổi này

- Viết LoRA đúng shape và scale.
- Tính số tham số và nhận ra giới hạn của rank.
- Kiểm gradient và phép merge bằng đường độc lập.
- Chèn adapter vào Transformer, so với full trên cùng giao thức.

01

Hai ma trận nhỏ

Nhánh gốc cộng nhánh sửa



LoRA giữ W₀ cố định và học phần sửa (α/r)·BA từ hai ma trận mỏng: với lớp 6×8 và r = 2, B có 12 số, A có 16 số.

Công thức của bài

$$\mathbf{y} = W_0\mathbf{x} + \frac{\alpha}{r} BA\mathbf{x}$$

$$W_0 : (d_{out}, d_{in}) \cdot A : (r, d_{in}) \cdot B : (d_{out}, r).$$

BA có hạng tối đa r.

HỎI NHANH

Với $d_{in} = 8$, $d_{out} = 6$, $r = 2$, hai ma trận A và B có shape gì?

A A là $(8, 2)$, B là $(2, 6)$

B A là $(2, 8)$, B là $(6, 2)$

C A là $(2, 6)$, B là $(8, 2)$

D Cả hai đều là $(6, 8)$, cùng shape với W_0

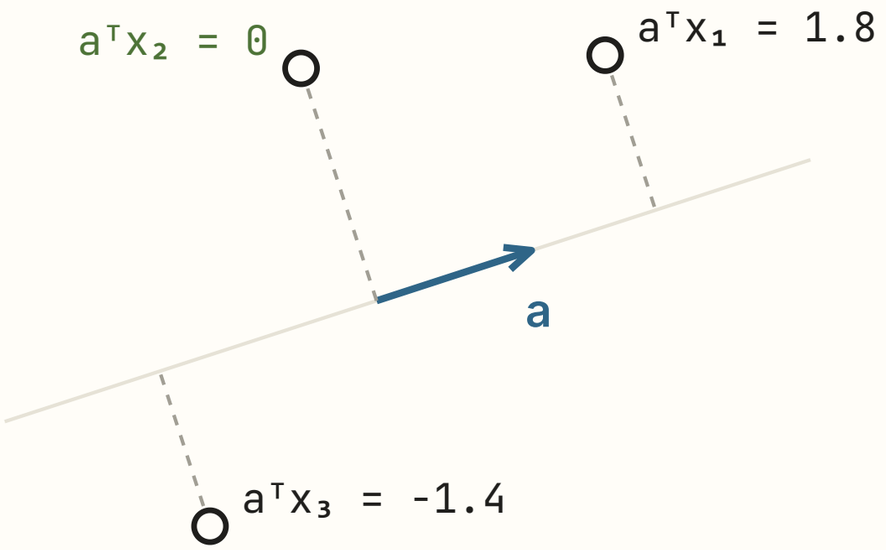
Rank 1: đo một hướng, đẩy một hướng

$$\frac{\alpha}{r} B A \mathbf{x} = \frac{\alpha}{r} \mathbf{b} \left(\mathbf{a}^\top \mathbf{x} \right)$$

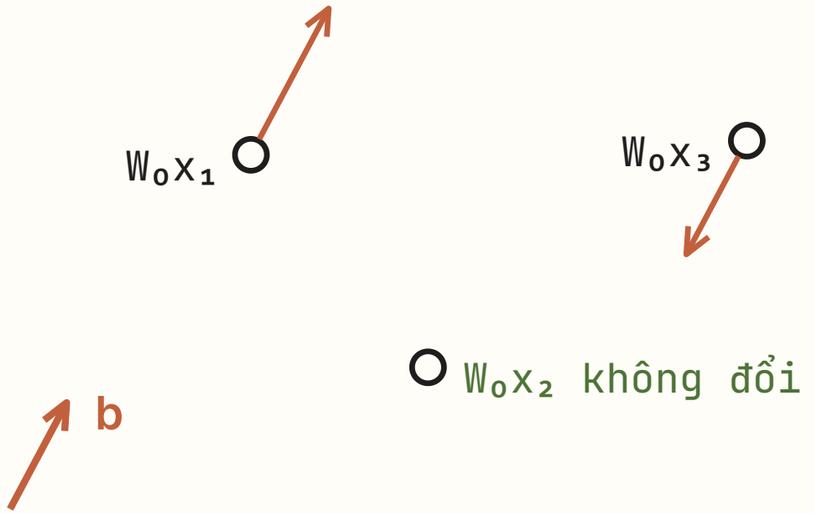
$\mathbf{a}$ chọn phần input được “nhìn thấy”; $\mathbf{b}$ là hướng sửa output.

Rank r là tổng của r cặp như vậy.

Một hướng nhìn, một hướng sửa



input: chỉ nhìn một hướng a



output: chỉ sửa dọc hướng b

LoRA rank 1: adapter đo aᵀx rồi đẩy output dọc đúng một hướng b, tỉ lệ với con số ấy; input vuông góc với a đi qua như model gốc.

HỎI NHANH

LoRA rank 1 có $A = \mathbf{a}^\top$ và $B = \mathbf{b}$. Một input $\mathbf{x}$ vuông góc với $\mathbf{a}$, tức $\mathbf{a}^\top \mathbf{x} = 0$. Adapter làm output của input này thay đổi thế nào?

A Cộng thêm $\frac{\alpha}{r}\mathbf{b}$, như một bias chung

B Đổi output theo hướng $\mathbf{a}$ thay vì $\mathbf{b}$

C Đổi mọi thành phần, vì BA có đủ shape (d_{out}, d_{in})

D Không đổi gì, output vẫn đúng bằng $W_0\mathbf{x}$

Tính tham số

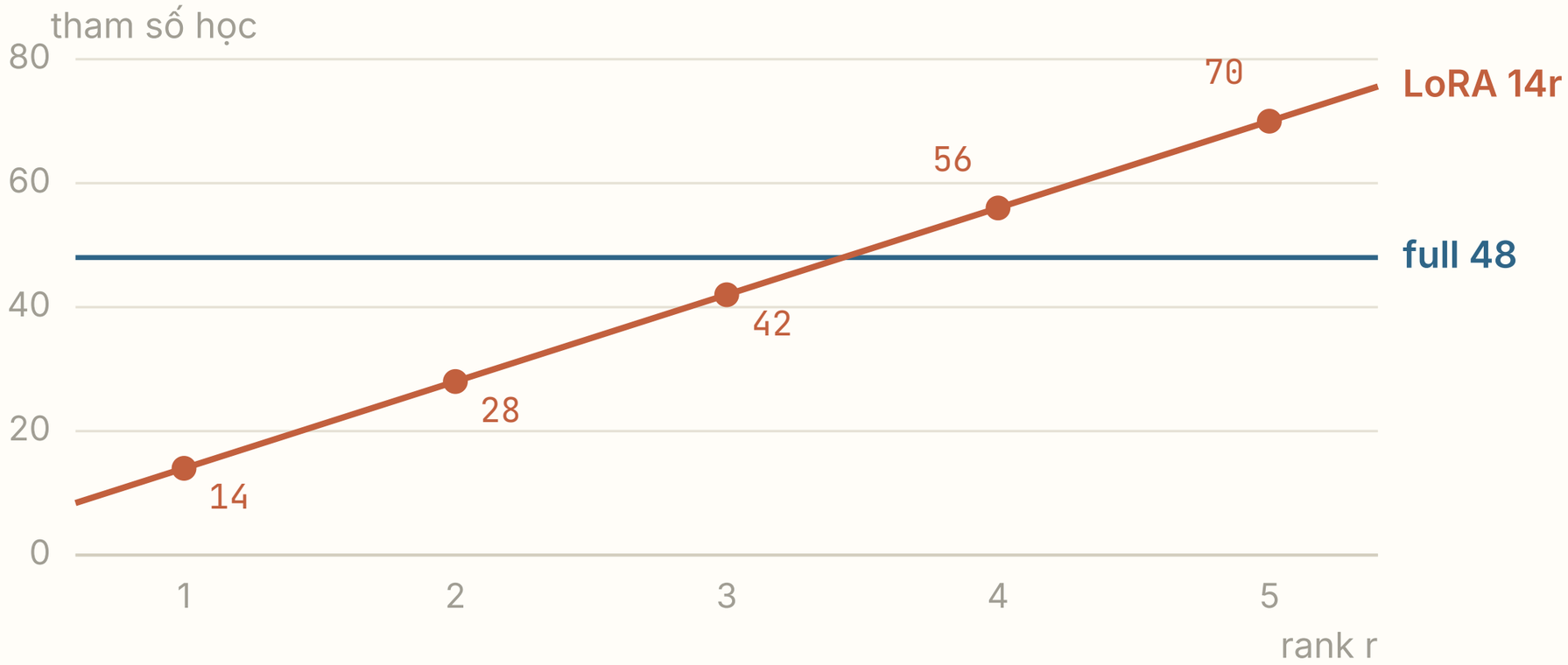
$$P_{full} = d_{out} d_{in}$$

$$P_{LoRA} = r(d_{in} + d_{out})$$

Lớp giả định	Full	LoRA
6×8, $r = 2$	48	28
4096×4096, $r = 8$	16 777 216	65 536

Tỉ lệ tham số học không phải tỉ lệ tổng bộ nhớ GPU.

Rank quá lớn có thể mất lợi thế



Lớp 6×8: LoRA học 14r tham số, vượt 48 của full update từ r = 4.

HỎI NHANH

DistilBERT có 6 block, mỗi block có hai lớp `q_lin` và `v_lin`, cùng là Linear 768 → 768. Chèn LoRA rank 4 vào cả hai lớp ở mọi block. Không tính head, adapter có bao nhiêu tham số?

A 6144

B 36 864

C 73 728

D 7 077 888

02

Khởi tạo và gradient

A ngẫu nhiên, B bằng 0

```
self.A = nn.Parameter(
    torch.randn(rank, din) * 0.1)
self.B = nn.Parameter(torch.zeros(dout, rank))
self.scale = alpha / rank
```

$BA = 0$ nên output ban đầu đúng model gốc.

Dữ liệu sinh từ W_0 + phần sửa hạng 2: biết trước đáp án.

HỎI NHANH

Lớp LoRA khởi tạo A ngẫu nhiên và $B = 0$. Vì sao output ban đầu đúng bằng output của lớp gốc?

A $BA = 0$ nên nhánh sửa chưa đóng góp gì

B W_0 bị freeze nên chưa tham gia forward

C Rank thấp làm phần sửa nhỏ, xấp xỉ bằng 0

D Scale α/r bằng 0 khi chưa train

Forward cho batch hàng

```
original = X @ self.base.T
update = (X @ self.A.T) @ self.B.T
return original + self.scale * update
```

KẾT QUẢ

```
ban dau dung model goc: True
tham so hoc: 28
```

Input `(N,8)` cho output `(N,6)`.

Chain rule chỉ đường học

$$G = \partial L/\partial Y,\ s = \alpha/r:$$

$$\frac{\partial L}{\partial B} = sG^\top X A^\top$$

$$\frac{\partial L}{\partial A} = sB^\top G^\top X$$

$B = 0$ nên gradient của A bằng 0 ở bước đầu; B vẫn học được.

HỎI NHANH

Với $N = 64$, $d_{in} = 8$, $d_{out} = 6$, $r = 2$, tích $G^\top X A^\top$ trong công thức gradient của B có shape gì?

A $(6, 2)$

B $(2, 6)$

C $(6, 8)$

D $(64, 2)$

HỎI NHANH

Ở bước đầu, với $B = 0$ và A ngẫu nhiên, autograd báo gradient của A bằng 0. Nên kết luận gì?

A Có lỗi, vì A chưa được đưa vào optimizer

B LoRA sẽ không bao giờ học được ma trận A

C Phải đặt $A = B = 0$ để hai ma trận cân bằng

D Đúng như chain rule, vì gradient của A chứa $B = 0$

Ba cách kiểm gradient

1. Autograd PyTorch.
2. Công thức NumPy.
3. Sai phân trung tâm cho B[0,0].

KẾT QUẢ

```
A grad bang 0: True
B grad khac 0: True
W0 grad None: True
khop cong thuc A/B: True True
khop sai phan B00: True
```

Kiểm A sau khi B đã học

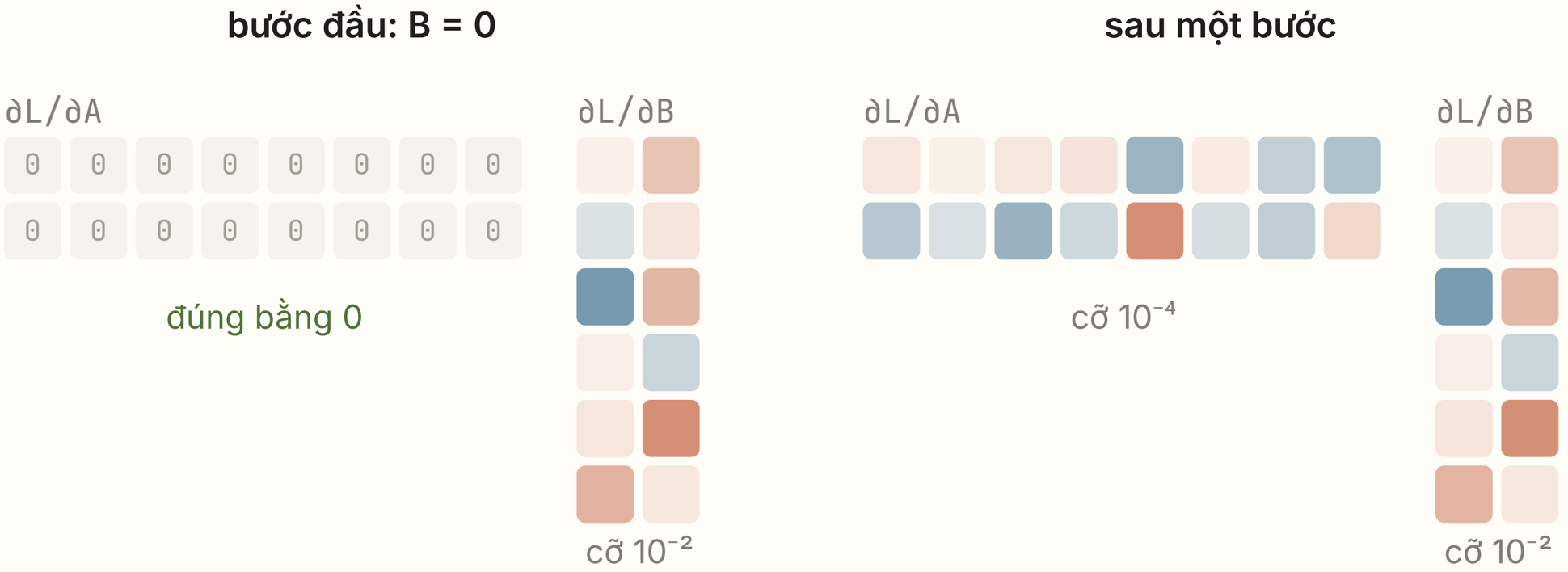
Ở $B = 0$, một bản gradient A luôn trả 0 cũng vượt được phép kiểm đầu.

Đi một bước để B khác 0, rồi kiểm lại A.

KẾT QUẢ

```
A grad sau buoc khac 0: True
A khop NumPy: True
A00 khop sai phan: True
```

Gradient ở hai trạng thái



Gradient ở bước đầu (số từ code bài): ∂L/∂A chứa B = 0 nên bằng 0, ∂L/∂B thì không; sau một bước, A bắt đầu nhận gradient. Màu theo dấu, đậm theo độ lớn trong từng ma trận.

03

Chất lượng và đóng gói

So trên một mục tiêu có rank tối đa 2

Cùng W_0, train/validation, Adam rate 0.03, $\alpha = 2r$.

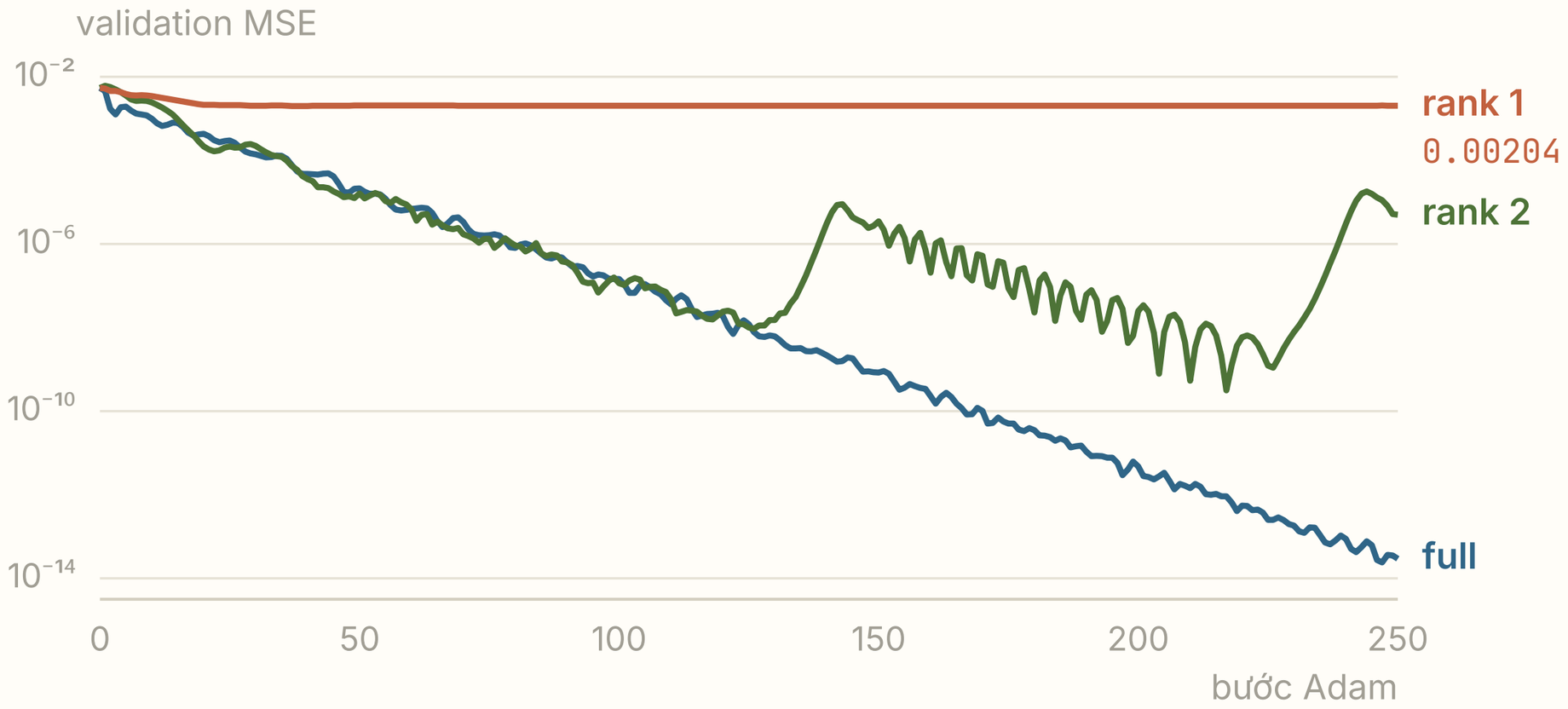
Mode	Tham số học	Val MSE
Full	48	2.93e-14
Rank 1	14	0.00204
Rank 1, 2500 bước	14	0.00204
Rank 2	28	5.04e-06

Rank 1 chạm sàn: thêm bước không giúp.

Rank 2 biểu diễn được, nhưng MSE dao động: con số phụ thuộc lúc dừng.

Chung ngân sách, không dò learning rate riêng.

Validation MSE theo bước

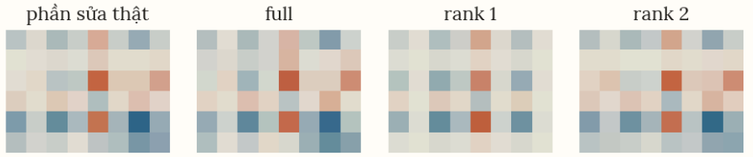


Validation MSE theo bước (số từ code bài, trục log): full về $3 \cdot 10^{-14}$; rank 2 có lúc xuống dưới 10^{-9} nhưng dao động, dừng ở $5 \cdot 10^{-6}$; rank 1 nằm ở sàn 0.00204 từ khoảng bước 40.

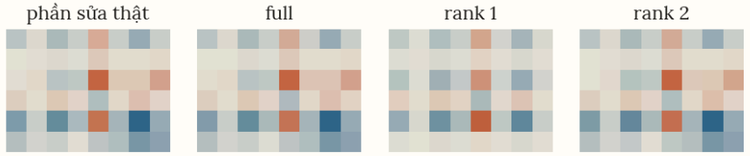
Phần sửa ΔW qua từng bước



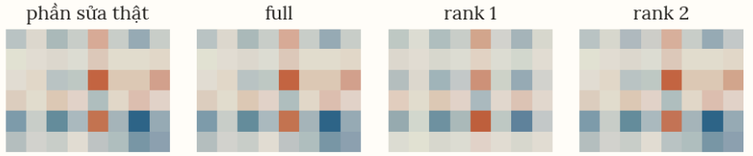
bước 0: validation MSE full 0.0054, rank 1 0.0054, rank 2 0.0054



bước 20: validation MSE full 0.00043, rank 1 0.0021, rank 2 0.00022



bước 70: validation MSE full 3.3e-06, rank 1 0.002, rank 2 1.7e-06



bước 250: validation MSE full 2.9e-14, rank 1 0.002, rank 2 5e-06

Thí nghiệm của bài: phần sửa ΔW mà full update, LoRA rank 1 và rank 2 học được sau từng số bước; ô cam dương, ô xanh âm.

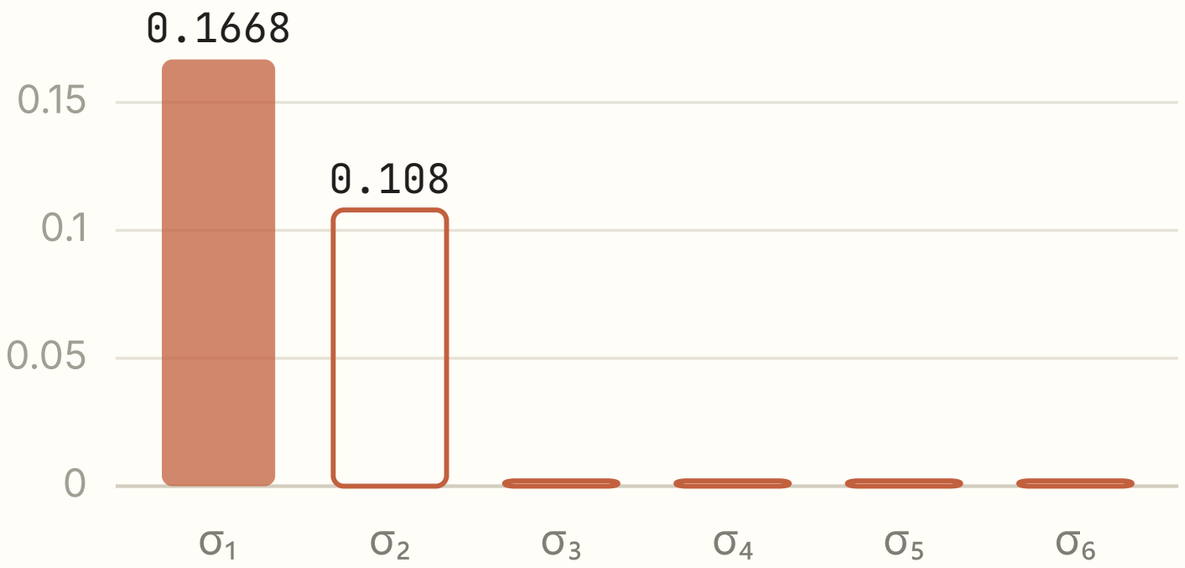
Sàn của rank 1

Giá trị kỳ dị của phần sửa: $0.1668, 0.108, 0, \ldots$

Rank 1 bỏ hướng thứ hai: MSE $\approx 0.108^2/6 \approx 0.0019$.

Sai số của rank r do các giá trị kỳ dị từ thứ $r+1$ quyết định.

Phổ của phần sửa



rank 1 giữ σ_1, bỏ σ_2

$\sigma_2^2 / 6 = 0.108^2 / 6$

≈ 0.0019

Giá trị kỳ dị của phần sửa thật (số từ code bài): chỉ hai giá trị khác 0. Rank 1 giữ σ_1, phần σ_2 bị bỏ góp khoảng $0.108^2/6 \approx 0.0019$ vào MSE.

HỎI NHANH

Code đặt $\alpha = 2r$, và recipe cũng vậy. Tăng rank từ 4 lên 16 thì hệ số scale α/r thay đổi thế nào?

A Tăng 4 lần, vì α tăng 4 lần

B Giảm 4 lần, vì chia cho r lớn hơn

C Phụ thuộc vào d_{in} và d_{out} của lớp

D Giữ nguyên bằng 2 với mọi rank

HỎI NHANH

Dựng lại thí nghiệm, cùng input, nhưng phần sửa có hai giá trị kỳ dị là 0.1668 và 0.01, các giá trị khác bằng 0. Train rank 1 đủ lâu thì validation MSE khoảng bao nhiêu?

A Vẫn cỡ 0.002, vì phần sửa vẫn có hạng 2

B Cỡ $2 \cdot 10^{-4}$, vì sàn giảm tỉ lệ với σ_2

C Cỡ $2 \cdot 10^{-5}$, vì sàn giảm theo σ_2^2

D Bằng 0, vì rank 1 đã giữ được hướng mạnh nhất

Merge để suy luận

$$W = W_0 + \frac{\alpha}{r} BA$$

Lớp tuyến tính của bài, không dropout, không quantization:

KẾT QUẢ

```
merge khop: True
rank delta: 2
```

HỎI NHANH

Đội có một base và hai adapter học riêng trên base ấy: X để phân loại chủ đề đề bài, Y để gắn nhãn độ khó. Cần phục vụ cả hai nhiệm vụ. Làm thế nào cho đúng?

A Giữ nguyên W_0, gắn X hoặc Y tùy nhiệm vụ

B Merge cả X và Y vào cùng W_0 rồi dùng chung

C Cộng $B_X + B_Y$ và $A_X + A_Y$ thành một adapter

D Chỉ giữ adapter có validation tốt hơn, dùng cho cả hai

Adapter cần base đúng phiên bản

Lưu A/B + cấu hình + head mới cần học.

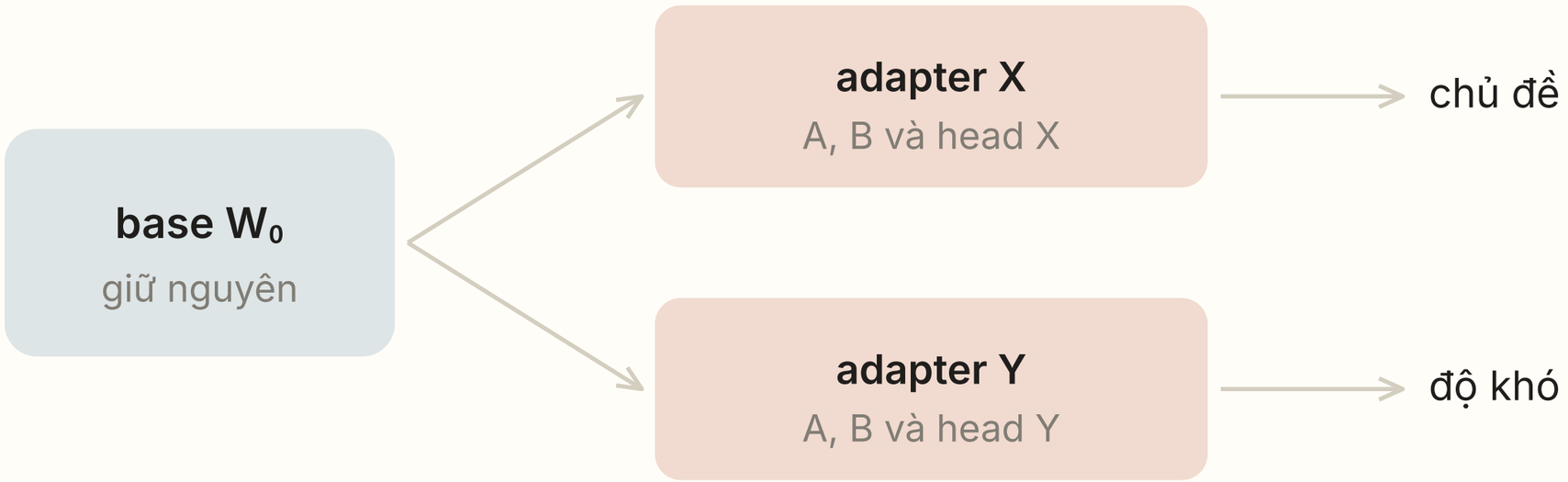
Ghi model ID, revision và hash.

Thiếu W_0 thì chưa có đủ model.

W_0 khác revision có thể cùng shape nhưng khác output.

Tải lại và so logits trước khi nộp.

Một base, nhiều adapter



Một base, mỗi nhiệm vụ một adapter kèm head riêng: đổi adapter là đổi nhiệm vụ, không merge hai adapter vào cùng một W_0.

04

LoRA trong model thật

Chèn LoRA vào DistilBERT

```
model.requires_grad_(False)
for name in targets:  # q_lin, v_lin
    parent_name, child_name = name.rsplit(".", 1)
    parent = model.get_submodule(parent_name)
    base = getattr(parent, child_name)
    setattr(parent, child_name,
            LoRALinear(base, rank=4, alpha=8))
model.pre_classifier.requires_grad_(True)
model.classifier.requires_grad_(True)
```

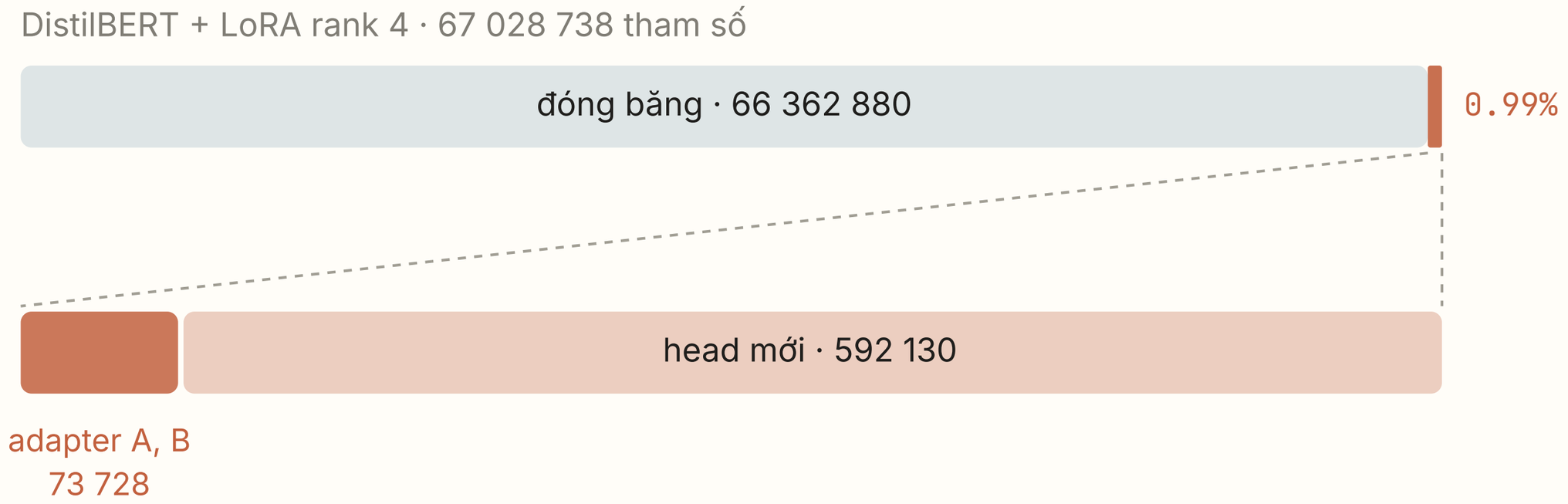
KẾT QUẢ

```
module duoc chen: 12
adapter: 73728 head: 592130
trainable: 665858 / 67028738 = 0.99%
logits giu nguyen: True
```

Trọng số ngẫu nhiên, đúng kiến trúc và tên module.

Head mới gấp 8 lần adapter.

Phần được học



Phần được học khi chèn LoRA rank 4 vào q_lin, v_lin của DistilBERT (số từ output bài): khoảng 1% tổng tham số, trong đó head mới gấp tám lần adapter.

HỎI NHANH

Một báo cáo ghi: “LoRA rank 4 trên DistilBERT chỉ học 0.11% tham số.” Với cấu hình như code trên, head phân loại mới cũng được học. Nhận định nào đúng?

A Đúng, vì head đã có sẵn trong checkpoint pretrained

B Đúng, vì head chỉ có 1538 tham số của `classifier`

C Thiếu head, thật ra khoảng 0.99%, vì head gấp 8 lần adapter

D Sai, phải tính cả 67 triệu vì đều nằm trên đường backward

Recipe PEFT với checkpoint thật

`text_experiment.py`: cùng CSV, bốn mode.

`features` · `head` · `full` · `lora`

PEFT bọc `q_lin`, `v_lin`, $\alpha = 2r$; head lưu qua `modules_to_save`.

`RESULT.json` của mỗi mode:

- Số tham số học.
- Macro-F1 validation.
- Thời gian chạy.
- Peak bộ nhớ khi chạy CUDA.

LoRA đáng giữ khi vượt head-only, gần full và rẻ hơn.

HỎI NHANH

LoRA làm số tham số học của một lớp chỉ còn 1/256. Đỉnh bộ nhớ GPU khi train có giảm còn 1/256 không?

A Có, vì bộ nhớ khi train chủ yếu là gradient

B Không, vì vẫn còn trọng số gốc và activation

C Không, vì LoRA luôn tốn bộ nhớ hơn học toàn bộ

D Có, miễn là rank là một lũy thừa của 2

Bẫy cần kiểm

- `no_grad()` quanh A/B cắt đường học.
- Tên target module sai → chèn 0 lớp, không báo lỗi.
- Quên cộng head/bias vào trainable.
- Rank lớn: $r(d_{in} + d_{out})$ có thể vượt $d_{in}d_{out}$.

HỎI NHANH

Một script tự viết chèn LoRA vào những lớp có tên kết thúc bằng `query` và `value`, tức tên trong BERT. Chạy trên DistilBERT như mục trước, số tham số trainable in ra là 592 130. Chuyện gì đã xảy ra?

A Không lớp nào khớp tên, nên chỉ head được học

B Adapter chèn đủ, nhưng $B = 0$ nên không tính là trainable

C Rank quá nhỏ nên adapter bị làm tròn về 0

D Head bị freeze nhầm, còn adapter đã chèn đủ

Khi đi thi

- Một batch: số module, trainable, logits lúc đầu, gradient.
- Baseline head-only, cùng split và ngân sách.
- Rank nhỏ trước; chọn bằng validation.
- Đo peak memory, thời gian trên đúng máy.
- Lưu adapter + head + base revision; tải lại, so logits.

05

Luyện đề

LUYỆN ĐỀ 1

Đội cần thích nghi một model ngôn ngữ lớn cho bài thi, chỉ với một GPU. Một bạn đề xuất LoRA, và lập luận thêm: “Model nộp sẽ dự đoán nhanh hơn, vì chỉ có vài trăm nghìn tham số.” Nhận định nào đúng?

A Đúng, vì lúc dự đoán chỉ cần A và B nên nhanh hơn base nhiều

B LoRA làm fine-tune rẻ hơn, nhưng lúc dự đoán input vẫn đi qua toàn bộ base

C LoRA cho phép dùng base lớn hơn mà không tốn thêm bộ nhớ khi suy luận

D LoRA tạo thêm dữ liệu train nên bớt công gán nhãn

PHỎNG THEO NOAI 2026 Singapore · Vòng sơ loại · Câu 246

LUYỆN ĐỀ 2

Lớp Linear $50 \to 100$, đặt LoRA $r = 64$. Nhận định nào đúng?

A Hạng tối đa 50, adapter 9600 tham số, nhiều hơn 5000 của học toàn bộ

B Hạng tối đa 64, adapter 9600 tham số, nhiều hơn 5000 của học toàn bộ

C Hạng tối đa 50, adapter vẫn ít tham số hơn học toàn bộ

D Hạng tối đa 64, adapter vẫn ít tham số hơn học toàn bộ

PHỎNG THEO NOAI 2026 Singapore · Vòng sơ loại · Câu 30

LUYỆN ĐỀ 3

Nén ΔW của full fine-tune thành rank 8 sao cho sai số Frobenius nhỏ nhất. Làm thế nào?

A Giữ 8 hàng dài nhất của ΔW

B Lấy 8 cột đầu làm B, 8 hàng đầu làm A

C SVD rồi giữ 8 giá trị kỳ dị nhỏ nhất

D SVD rồi giữ 8 giá trị kỳ dị lớn nhất

PHỎNG THEO USA-NA-AIO 2026 · Vòng 1 · Part 5.14

Điều cần giữ lại

- LoRA học sBA và giữ W_0; hạng không quá r.
- Khởi tạo giữ output nguồn nhưng vẫn mở đường gradient.
- Đếm cả adapter lẫn head; kiểm gradient, merge, rồi đo thực nghiệm.

Sau giờ học

Làm **10 câu kiểm tra cuối bài** trên trang bài học.

Chạy notebook; thử rank 1–3 và đọc sàn MSE.

Đọc recipe trước khi áp LoRA cho checkpoint thật.